Brickts SQL course — Week 4 — Write safe application SQL

sparql-playground / lessons/sql

Generated for offline / Kindle reading

Week 4 — Write safe application SQL

This pack concatenates the six daily lessons and the weekly challenge, then the Week primary-reading links from SOURCES.md. Work the SQL in `psql` against `brickts_course`; keep notes under ignored `student-work/sql/`.

Contents

- Day 22: Day 22 — Observe commits, rollbacks, and a competing writer
- Day 23: Day 23 — Bind query values with SQL PREPARE
- Day 24: Day 24 — Change a schema with a reviewable migration
- Day 25: Day 25 — Inspect privileges and the limits of a row policy
- Day 26: Day 26 — Read an execution plan and justify an index

- Day 27: Day 27 — Turn assumptions into reviewable SQL checks
- Day 28 (weekly challenge): Day 28 — Week 4 challenge — a reviewable ingest schema

---

Day 22 — Observe commits, rollbacks, and a competing writer

Previous: Day 21 · **Week 4 · 45–75 minutes.**
Prerequisites: days 1–21, or equivalent skills.

Goal

Observe transaction visibility from two clients and explain why concurrent writes need coordination. Keep every experiment confined to one scratch row with a short lock timeout.

Before you start

Open two terminals, each connected to the same `brickts_course` database through the lab helper. Label them A and B in your notes. Confirm both connections with `\conninfo`. Do not run this exercise on an application database or shared operational table.

Concept

A transaction's uncommitted changes are visible to that transaction but ordinarily not to another session. PostgreSQL's usual default isolation level, Read

Committed, gives each statement a view of committed data as it begins. A competing update can wait for another writer's row lock.

An atomic expression such as `value = value + 1` lets the database perform an increment against the row it updates. Reading a value into a client and later assigning a computed constant can overwrite another writer's work unless the design adds a suitable concurrency strategy. Rollback removes uncommitted changes and releases locks; disconnecting an unfinished session also aborts its open transaction.

Tiny example

In A, create a one-row scratch table outside an explicit transaction:

```
CREATE TABLE student.day22_counter (
    counter_id integer PRIMARY KEY CHECK
         (counter_id = 1),
    value integer NOT NULL
);
INSERT INTO student.day22_counter (counter_id,
         value) VALUES (1, 0);
BEGIN;
UPDATE student.day22_counter SET value = value + 1
         WHERE counter_id = 1;
SELECT value FROM student.day22_counter WHERE
         counter_id = 1;
```

A sees `1`. Before A commits, B's plain `SELECT` sees `0`. In A, run `ROLLBACK;`; both sessions then see `0`.

SQL challenge

Repeat the increment and commit it instead. Verify both sessions see `1` after commit. Record a small timeline containing the statement, session, transaction

state, and observed value.

Test contention: while A holds an uncommitted update, start a transaction in B, run `SET LOCAL lock_timeout = '2s';`, then attempt an update of the same row. Record the timeout, run `ROLLBACK;` in B, and finish A with rollback. Every open transaction must be closed before you leave.

Experiment

Run `SHOW transaction_isolation;`. Explain your observed visibility using that result rather than assuming all isolation levels behave alike. Compare the atomic increment with two clients independently reading `1` and later assigning `2`; predict the lost-update risk without implementing a larger race.

Acceptance checks

- Uncommitted and committed visibility have distinct recorded evidence.
- The competing write stops within the declared timeout, and both clients recover.
- The scratch counter ends at a known committed value with no transaction left open.

Stretch and reflection

Read week 4 references about isolation and explicit locking. When might `SELECT ... FOR UPDATE` be useful, and what extra waiting behavior would it introduce? Keep your answer conceptual until a bounded test is designed.

Previous: Day 21 ·

Day 23 — Bind query values with SQL PREPARE

Previous: Day 22 · **Week 4 · 45–75 minutes.**
Prerequisites: days 1–22, or equivalent skills.

Goal

Separate query structure from supplied values using PostgreSQL's SQL-level prepared statements. No Python client or application framework is required today.

Before you start

Connect to the course database in a fresh `psql` session. Recall that single quotes mark text values and that point identity is a stored UUID. The prepared statements in this lesson exist only in your current session.

Concept

A parameter is a value supplied to a query whose structure is already specified. `PREPARE` declares parameter types and uses placeholders such as `$1`. `EXECUTE` supplies values for those positions. This keeps input text in the value position, rather than asking a program to concatenate it into SQL syntax.

Parameters stand for values, not table names, column names, or arbitrary ordering clauses. An application that genuinely needs dynamic identifiers must select

from an explicit allowlist and use the database driver's identifier mechanisms. Driver parameter APIs have their own placeholder syntax; today's SQL syntax teaches the boundary without assuming a particular language.

Tiny example

```
PREPARE day23_equipment (text) AS
SELECT equipment_id, site_id, label, kind
FROM course.equipment
WHERE site_id = $1
ORDER BY equipment_id;

EXECUTE day23_equipment ('SYN-NORTH');
EXECUTE day23_equipment ('SYN-SOUTH');
```

The executions return two and one equipment rows respectively. The query structure stays the same while the site value changes. Use `DEALLOCATE day23_equipment;` when you finish or simply close the session.

SQL challenge

Prepare a bounded sample-read statement accepting a site ID, point UUID, start instant, and end instant. Use an explicit half-open window, show timestamp/value/quality, and order by time. Include a fixed small result limit for inspection. Retrieve the chosen point UUID from metadata and record its site, role, and unit separately; never derive a UUID from a label.

Execute the statement for two adjacent windows of that same point. Compare each result to the equivalent literal query. State what should happen for a correctly

typed but nonexistent point UUID and for a malformed UUID value.

Experiment

Before deallocating the example, run:

```
EXECUTE day23_equipment ('SYN-NORTH'' OR true --');
```

It returns zero rows because the entire argument is one unmatched site value. The embedded apostrophe is escaped in the SQL literal. Explain why the characters are data here. Do not test by building an unsafe string-concatenation client.

Acceptance checks

- Prepared inventory results contain 2 northern and 1 southern rows.
- The sample query binds four typed values and retains the quality and time-window policy.
- Suspicious input remains a value, and your explanation distinguishes value parameters from identifiers.

Stretch and reflection

Inspect `pg_prepared_statements` in the current session. What disappears when the session ends? Read week 4 references for prepared statements and your eventual driver's parameter interface.

Previous: Day 22 ·

Day 24 — Change a schema with a reviewable migration

Previous: Day 23 · **Week 4 · 45–75 minutes.**
Prerequisites: days 1–23, or equivalent skills.

Goal

Write a small scratch migration with preconditions, a bounded backfill, and postconditions. Separate the meaning of existing data from defaults for future writes.

Before you start

Create only `student` objects. A migration is a saved, ordered change to a schema, often accompanied by data changes. Today's exercise uses a few scratch rows; it does not migrate the course fixture or an application database.

Concept

Adding a required column to existing data needs a policy for those existing rows. A default describes what happens when a future insert omits a value. It does not prove a historical fact. A backfill assigns values to existing rows under a declared rule. Fabricating observation timestamps or measured values to satisfy a constraint would misrepresent evidence.

A migration should name its starting assumption and desired final state. PostgreSQL can transact many DDL changes, so a small migration can be tested and rolled

back as a unit. Schema changes acquire locks and may affect other clients; a successful tiny lab is not evidence of safe timing on a large shared table. A rollout also needs to consider old and new writers.

Tiny example

```
CREATE TABLE student.day24_batch (
    batch_id text PRIMARY KEY,
    source_label text NOT NULL
);
INSERT INTO student.day24_batch (batch_id,
         source_label)
VALUES ('practice-batch-1', 'synthetic excerpt');

BEGIN;
ALTER TABLE student.day24_batch ADD COLUMN
         review_state text;
UPDATE student.day24_batch
SET review_state = 'pending'
WHERE review_state IS NULL;
ALTER TABLE student.day24_batch
    ALTER COLUMN review_state SET DEFAULT
         'pending',
    ALTER COLUMN review_state SET NOT NULL;
ALTER TABLE student.day24_batch
    ADD CONSTRAINT day24_review_state_check
    CHECK (review_state IN ('pending',
         'reviewed'));
COMMIT;
```

One row remains, now pending review. That value is a declared workflow policy, not an inferred historical measurement.

SQL challenge

Design your own two-row scratch metadata table and a migration adding a required review flag or state. Document why the chosen backfill is truthful. Save the

original DDL, migration, precondition queries, and postcondition queries separately so another learner can review them.

Verify unchanged row count and identities, complete backfill, accepted future default behavior, and rejection of an invalid state. Include a short note about how an old writer that omits the new column behaves.

Experiment

On a fresh scratch table, rehearse a similar migration inside `BEGIN` and inspect the changed definition before running `ROLLBACK`. Confirm both schema and data return to the starting state. Explain why rollback testing is easiest before other code begins relying on the new shape.

Acceptance checks

- Your migration has explicit starting and ending contracts.
- Existing identities and row count survive, and no measurement facts are invented.
- Backfill behavior, future defaults, invalid input, and rollback have separate evidence.

Stretch and reflection

What makes re-running a migration different from retrying one ingest batch? Describe how a migration ledger could prevent accidental repeat execution. Read week 4 references for `ALTER TABLE`, defaults, transactions, and locks.

Previous: Day 23 ·

Day 25 — Inspect privileges and the limits of a row policy

Previous: Day 24 · **Week 4 · 45–75 minutes.**
Prerequisites: days 1–24, or equivalent skills.

Goal

Explain database roles, object privileges, and row-level security, or RLS, while inspecting your own disposable lab connection. No server-wide role creation or administrative grant changes are required.

Before you start

Connect to `brickts_course` and record `SELECT current_user, session_user;`. Credentials stay outside your notes. The lab connection may own its objects or have broad privileges; that is convenient for learning, but it is not evidence that an application reader has restricted access.

Concept

A role can own objects and receive privileges. A schema needs `USAGE` for objects inside it to be reachable; table `SELECT` grants reading, while `INSERT`, `UPDATE`, and `DELETE` allow different writes. Ownership and membership affect the effective rights. A grant on one table does not automatically establish the desired policy for all future tables.

RLS adds row conditions to applicable queries; it does not replace ordinary object privileges. Superusers and roles with BYPASSRLS bypass policies. Table owners normally bypass RLS unless forced, and even forcing it does not constrain a superuser. Testing as the owner therefore cannot establish a restricted user's isolation. A caller-controlled site setting is also not authenticated site ownership.

Tiny example

```
SELECT current_user,
       has_schema_privilege(current_user, 'course',
         'USAGE') AS course_usage,
       has_table_privilege(current_user,
         'course.sample', 'SELECT') AS
         can_read_samples;

SELECT rolname, rolsuper, rolbypassrls
FROM pg_catalog.pg_roles
WHERE rolname = current_user;
```

A working fixture reader should report true for schema usage and sample reading. The other flags depend on the actual lab role; record them rather than inventing a result.

A policy definition on a scratch object uses this syntax:

```
CREATE TABLE student.day25_note (
    note_id integer PRIMARY KEY,
    site_id text NOT NULL,
    note text NOT NULL
);
ALTER TABLE student.day25_note ENABLE ROW LEVEL
         SECURITY;
CREATE POLICY day25_north_read ON
         student.day25_note
    FOR SELECT USING (site_id = 'SYN-NORTH');
SELECT row_security_active('student.day25_note'::regclass)
         AS policy_applies;
```

As this table's owner, you normally get `false`: the policy exists but does not restrict your current reads. Do not confuse successful policy creation with a successful isolation test.

SQL challenge

Write a privilege matrix for three conceptual roles: metadata reader, sample ingester, and migration owner. For each, name required schema/table actions and explain any ownership. These are design notes, not commands to alter server roles.

Populate the scratch notes table with one explicitly synthetic note for each site. Inspect its policy through `pg_catalog.pg_policies`, limiting the result to the `student` schema and this table. Read the notes and compare their visibility with `row_security_active`. Explain why the observed output does or does not establish isolation for an intended reader. If your role bypasses the policy, document that limitation instead of claiming isolation.

Experiment

Compare the observed privilege booleans with your successful and failed operations from earlier lessons. Explain why a zero-row result can come from a filter, an RLS policy, or genuinely absent data, while a permission error is a different outcome. Avoid changing shared roles to manufacture a result.

Acceptance checks

- Your evidence identifies the current role's read privilege and bypass flags without exposing credentials.
- Your matrix assigns only the actions each role needs.
- Your RLS note explicitly states whether enforcement was active for the testing role.

Stretch and reflection

What separate test identity would be needed to verify two-site isolation? Describe the expected allowed and denied cases for a DBA to provision. Read week 4 references for grants and RLS caveats.

Previous: Day 24 ·

Day 26 — Read an execution plan and justify an index

Previous: Day 25 · **Week 4 · 45–75 minutes.**
Prerequisites: days 1–25, or equivalent skills.

Goal

Read the basic shape of an `EXPLAIN` plan and evaluate one scratch index against a concrete access pattern. Preserve query correctness while investigating performance.

Before you start

Use the disposable course database. `EXPLAIN` describes a planned execution; `EXPLAIN (ANALYZE, BUFFERS)` executes the statement and reports actual work. Today's analyzed statements are bounded `SELECT`s. Applying that option to a write would execute the write.

Concept

A sequential scan reads table rows and checks conditions. An index scan uses a separate access structure to find candidate rows, then may visit table pages. The planner estimates cost and row counts using statistics. Estimates are not milliseconds, and an index is not automatically faster for a small table or a query reading most rows.

A multicolumn B-tree index has an ordered key. Equality on site and point followed by a timestamp range matches the common point-history access pattern. The existing sample key already supports that pattern; avoid adding a duplicate index simply because a tutorial mentions indexing. Every extra index consumes storage and adds write work.

Tiny example

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT site_id, point_id, ts, value, quality
FROM course.sample
WHERE site_id = 'SYN-NORTH'
  AND ts >= TIMESTAMPTZ '2026-01-05 08:00:00+00'
  AND ts <  TIMESTAMPTZ '2026-01-05 09:00:00+00'
ORDER BY point_id, ts;
```

The actual result contains 334 rows: two AHUs × fourteen roles × twelve instants, minus two absent pressure rows. Scan and sort choices depend on the server and statistics. Record the plan you observe instead of requiring a particular node name.

SQL challenge

Create one bounded scratch copy of the fixture with `CREATE TABLE student.day26_sample AS SELECT ... FROM course.sample`. It has 24,190 rows and no copied primary key. Inspect its indexes, run `ANALYZE`, and capture a plan for a single-point half-open time-window lookup using a UUID retrieved from metadata.

Add one scratch index on `(site_id, point_id, ts)`, analyze again, and compare plans for the exact same query. Record result equality, estimated/actual row

counts, scan choice, and buffers. A plan that stays sequential can be a valid outcome.

Experiment

Compare the selective lookup with a query reading most scratch rows. Predict why the useful access strategy might change. Keep both histories bounded to the fixture; do not manufacture millions of rows or run repeated timing loops.

Acceptance checks

- The example's actual row count reconciles to 334.
- Before/after queries return identical data and your index exists only in `student`.
- You explain one estimated/actual difference or one planner choice without claiming universal speedup.

Stretch and reflection

Read week 4 references for `EXPLAIN`, statistics, and multicolumn indexes. What write cost and storage tradeoff accompanies the index you tested?

Previous: Day 25 ·

Day 27 — Turn assumptions into reviewable SQL checks

Previous: Day 26 · **Week 4 · 45–75 minutes.**
Prerequisites: days 1–26, or equivalent skills.

Goal

Write small SQL checks that distinguish fixture expectations from general data contracts, and collect evidence another person can interpret without guessing what “pass” means.

Before you start

Keep your schema notebook, join report, coverage report, and migration notes available. Today’s checks read `course` and may use scratch data for intentional failures. A failed course check should trigger investigation, not a change to the expected number just to obtain green output.

Concept

An invariant is a rule that must hold, such as a sample referencing a point in the same site. A fixture expectation describes this particular seed, such as having exactly forty-two points. Both are useful, but they have different lifetimes. Legitimate new equipment should change fixture totals while preserving relationship integrity.

A check needs a named question, expected outcome, actual outcome, and failure interpretation. Some checks return a Boolean row; others list violations and pass when that list is empty. Document which convention you use. A query that prints nothing because it failed to run is not a passing empty result.

Tiny example

```
WITH checks AS (
    SELECT 'site_count' AS check_name, 2::bigint AS
         expected,
           count(*) AS actual
    FROM course.site
    UNION ALL
    SELECT 'point_count', 42::bigint, count(*)
    FROM course.point
    UNION ALL
    SELECT 'sample_count', 24190::bigint, count(*)
    FROM course.sample
)
SELECT check_name, expected, actual, actual =
         expected AS passed
FROM checks
ORDER BY check_name;
```

All three fixture checks should return true after a fresh load. They prove totals, not correct joins, units, quality handling, or time support.

SQL challenge

Create a report-contract file with checks for inventory totals, the single bad/null value, uniqueness of the joined logical observation key, and site-consistent sample-to-point relationships. Include at least one check on a summary's grain or quality policy. State whether each check is fixture-specific or intended as a lasting invariant.

For join uniqueness, a violation query can group by `(site_id, point_id, ts)` and return groups with `count(*) > 1`. Run it against the same joined relation your report actually uses, not only the base table that already has a key.

Experiment

Use a small scratch dataset to make one contract check fail deliberately. Suitable cases include duplicate joined rows or a report population containing an unusable measurement. Record the failing evidence, fix the scratch report, and observe the check pass. Keep the fixture untouched.

Acceptance checks

- Your fixture totals are 2 sites, 42 points, and 24,190 samples.
- Every check has a clear expected outcome and explains its failure meaning.
- One deliberate scratch failure demonstrates that the checks can detect a real report mistake.

Stretch and reflection

Run `uv run python lessons/sql/lab.py check` from the repository root and keep its actual status with your SQL evidence. Explain what your report checks add beyond fixture loading checks. Read week 4 references for constraints and transaction behavior.

Previous: Day 26 ·

Day 28 — Week 4 challenge — a reviewable ingest schema

Previous: Day 27 · **Week 4 review · 2–4 hours.**
Prerequisites: days 22–27.

Project brief

Build a small, reviewable ingest design in `student` for synthetic point observations. Combine keys, ownership, quality, retry behavior, transactions, parameters, migrations, and contract evidence. Work independently; allow 2–4 hours and keep the dataset bounded to a small fixture excerpt.

The reviewer should be able to understand what a row means, why a retry is safe, which values are eligible for calculations, and what remains unknown. Preserve the original course fixtures. Your submission is a learning artifact, not a deployment to an operational database.

Acceptance criteria

- Tables have explicit grains, keys, types, units, quality rules, and site-consistent relationships; repeated labels do not become identity.
- Repeating one ingest batch creates no duplicate logical rows, and a correction policy is documented.
- A transaction demonstrates atomic success and rollback with compact actual evidence.
- A parameterized read uses point identity, a half-open UTC window, explicit order, and a bounded result.

- A saved scratch migration includes truthful backfill policy and pre/postconditions.
- A privilege design explains intended reader/writer access and any limits of the testing role's RLS evidence.
- An index choice follows an observed plan and a named access pattern.
- Checks cover both fixture facts and lasting relationships, including one deliberately failing scratch case.

Deliverables

Submit DDL, bounded ingest SQL, prepared-query SQL, migration, contract checks, and a short review note. Include actual outputs or mark a check not run. State at least one remaining limitation and a concrete next test; do not report an untested claim as verified.

Review questions

What happens after a retry, a bad late correction, or an interrupted transaction? Which ownership rule spans more than one column? What does your role permit that a reader should not receive? Which plan observation justifies an index?

Consult week 4 references after your independent design. Next week develops engineering calculations on top of these contracts.

Previous: Day 27 ·

Primary reading — Week 4

Use the lesson first, then these references for precise behavior. Full list: SOURCES.md.

Week 4

- Transactions
- Isolation
- PREPARE
- Roles
- Row security
- ALTER TABLE